

MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

LOS LAB – Assignment 5

Reg. Number	Name	Branch
261100690024	K P NIDHI	CyberSecurity



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

Stream Editing with sed for Security Log Analysis

1. Theoretical Background

sed is a stream editor. It reads a file line by line, applies whatever commands you give it, and prints the result to the screen. It does not change the original file unless you specifically tell it to with the -i option, which is what makes it safe to use on log files during an investigation.

Addresses

An address tells sed which lines to work on. It can be a line number (3), a range (2,6), the last line ($), a step (1~2 means every second line starting from line 1), or a pattern in slashes (/Failed password/).

Main Commands Used

p prints a line, d deletes it, s/old/new/ substitutes text, i inserts a line before, a appends a line after, c replaces the whole line, G adds a blank line, and w writes matching lines to a file. The -n option turns off sed's automatic printing, which you need whenever you use p, otherwise every line gets printed twice.

Sample Log Files

Since the question paper did not include the log contents, I created my own auth.log and access.log with entries covering everything the exercises ask about failed and successful logins, invalid users, sudo events, connection closures, and web requests with 200, 401 and 403 status codes, including path traversal, XSS and SQL injection attempts.

```
┌──(kali㉿kali)-[~/sed_lab]
└─$ cat -n auth.log
     1  Sep 15 08:01:12 server sshd[1001]: Failed password for root from 172.
16.0.8 port 54321 ssh2
     2  Sep 15 08:02:45 server sshd[1002]: Accepted password for admin from 1
92.168.1.15 port 54322 ssh2
     3  Sep 15 08:03:10 server sshd[1003]: Invalid user test from 10.0.0.25 p
ort 54323
     4  Sep 15 08:04:22 server sshd[1004]: Failed password for admin from 172
.16.0.8 port 54324 ssh2
     5  Sep 15 08:05:33 server sshd[1005]: Accepted password for student from
 192.168.1.15 port 54325 ssh2
     6  Sep 15 08:06:44 server sudo: pam_unix(sudo:auth): authentication fail
ure; logname=admin uid=1000
     7  Sep 15 08:07:55 server sshd[1006]: Connection closed by 10.0.0.25 por
t 54326
     8  Sep 15 08:08:12 server sshd[1007]: Invalid user oracle from 172.16.0.
8 port 54327
     9  Sep 15 08:09:30 server sudo: admin : TTY=pts/0 ; PWD=/home/admin ; US
ER=root ; COMMAND=/bin/cat
    10  Sep 15 08:10:41 server sshd[1008]: Failed password for student from 1
0.0.0.25 port 54328 ssh2
```

Figure 1: The sample auth.log file with 10 numbered entries.

Observation / Explanation: This is the authentication log I built for Part A. It has 3 failed passwords, 2 accepted passwords, 2 invalid user attempts, one authentication failure from sudo, one connection closed line, and one sudo command line. The IPs 172.16.0.8, 192.168.1.15 and 10.0.0.25 all appear because later exercises ask about them specifically.

```
┌──(kali㉿kali)-[~/sed_lab]
└─$ cat -n access.log
     1  10.0.0.25 - - [15/Sep/2026:08:01:12 +0000] "GET /index.html HTTP/1.1"
 200 1043
     2  172.16.0.8 - - [15/Sep/2026:08:02:45 +0000] "POST /admin/login HTTP/1
.1" 401 512
     3  192.168.1.15 - - [15/Sep/2026:08:03:10 +0000] "GET /images/logo.png H
TTP/1.1" 200 8734
     4  172.16.0.8 - - [15/Sep/2026:08:04:22 +0000] "GET /../../etc/passwd HT
TP/1.1" 403 298
     5  10.0.0.25 - - [15/Sep/2026:08:05:33 +0000] "GET /search?q=<script>ale
rt(1)</script> HTTP/1.1" 403 305
     6  172.16.0.8 - - [15/Sep/2026:08:06:44 +0000] "POST /login?id=1' OR '1'
='1 HTTP/1.1" 401 487
     7  192.168.1.15 - - [15/Sep/2026:08:07:55 +0000] "GET /about.html HTTP/1
.1" 200 2201
     8  10.0.0.25 - - [15/Sep/2026:08:08:12 +0000] "POST /admin/login HTTP/1.
1" 403 401
```

Figure 2: The sample access.log file with 8 numbered web requests.

Observation / Explanation: This is the web server log for Part B. It has three 200 responses, two 401s and three 403s. I deliberately put in a ../ path traversal attempt, a <script> XSS attempt, and an OR based SQL injection attempt so the pattern searches in Exercise 6 would actually find something.

2. Exercise 1 — Printing Entries

These commands print selected lines from auth.log using line numbers, ranges and patterns.

```
sed -n '1p' auth.log                                          # first entry
sed -n '5p' auth.log                                          # fifth entry
sed -n '2,6p' auth.log                                        # lines 2 to 6
sed -n '$p' auth.log                                          # last entry
sed -n '1~2p' auth.log                                        # every 2nd from line 1
sed -n '/Failed password/p' auth.log                          # failed logins
sed -n '/Accepted password/p' auth.log                        # successful logins
sed -n '/Failed password/,/authentication failure/p' auth.log # range between patterns
```

Observation / Explanation: The -n is important here. Without it sed prints every line by default, and then p prints the matched lines a second time, so you get duplicates. $ is a shortcut for the last line. 1~2 is the step form, so it gives lines 1, 3, 5, 7 and 9. The last command uses two patterns as a range, so it starts printing at the first failed password on line 1 and stops at the authentication failure on line 6.

3. Exercise 2 — Filtering Security Events

```
sed -n '/Failed password/p' auth.log                  # failed logins
sed -n '/Accepted password/p' auth.log                # successful logins
sed -n '/root/p' auth.log                             # root account
sed -n '/Invalid user/p' auth.log                     # invalid users
sed -n '/172\.16\.0\.8/p' auth.log                    # one specific IP
sed -n '/Accepted password/!p' auth.log               # NOT successful
sed -n '/sudo/!p' auth.log                            # everything except sudo
sed -n -e '/Failed password/p' -e '/Invalid user/p' auth.log   # two patterns
```

Observation / Explanation: The ! after a pattern reverses it, so /sudo/!p prints every line that does not mention sudo. I escaped the dots in the IP as \. because a plain dot in a regex means any

character, so without the backslashes it would also match things like 172x16y0z8. The last one uses two -e options to run two separate print commands in one pass.

4. Exercise 3 — Removing Irrelevant Entries

```
sed '/Accepted password/d' auth.log        # drop successful logins
sed '/Connection closed/d' auth.log        # drop connection closed
sed '1,3d' auth.log                        # drop first three
sed '4,7d' auth.log                        # drop lines 4 to 7
sed '/192\.168\.1\.15/d' auth.log          # drop that IP
sed '1~2d' auth.log                        # drop every 2nd from line 1
sed '/Failed password/!d' auth.log         # keep ONLY failed passwords
```

Observation / Explanation: None of these use -i, so auth.log itself is never touched. sed just prints the edited version to the screen and the file on disk stays exactly as it was, which is what the question asked for. The last command is a neat trick — instead of deleting what you don't want, it deletes everything that is not a failed password, so you end up keeping only those lines.

5. Exercise 4 — Classifying Security Events

```
sed 's/Accepted password/[SUCCESSFUL LOGIN]/' auth.log
sed 's/Failed password/[FAILED LOGIN]/' auth.log
sed 's/Invalid user/[INVALID USER]/' auth.log
sed 's/authentication failure/[AUTHENTICATION FAILURE]/' auth.log
sed 's/root/PRIVILEGED_USER/g' auth.log
sed 's/server/authentication-server/g' auth.log
```

Observation / Explanation: The s command works as s/what to find/what to replace it with/. I added g on the last two because root and server can show up more than once on the same line — line 9 has both admin and USER=root in it. Without the g, sed would only change the first match on each line and leave the rest.

6. Exercise 5 — Masking Sensitive Information

```
sed 's/10\.0\.0\.25/X.X.X.X/g' auth.log
sed 's/172\.16\.0\.8/SUSPICIOUS_IP/g' auth.log
sed 's/admin/USER_A/g' auth.log
sed 's/student/USER_B/g' auth.log
sed 's/[0-9]/X/g' auth.log                        # every digit

# periods in IPs changed to hyphens
sed -E 's/([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})/\1-\2-\3-\4/g' auth.log

# every IP replaced with a label
sed -E 's/([0-9]{1,3}\.){3}[0-9]{1,3}/[REDACTED_IP]/g' auth.log
```

Observation / Explanation: [0-9] matches any single digit, so with g it wipes out every number in the file including the timestamps and port numbers. For the hyphen one I used -E to turn on extended regex, which lets me write ( ) and {1,3} without backslashes. The four bracketed groups get captured and then referred back to as \1 \2 \3 \4 in the replacement, so each part of the IP is kept but the dots between them become hyphens.

7. Exercise 6 — Printing Web Requests

```
sed -n '/" 200 /p' access.log           # status 200
sed -n '/" 401 /p' access.log           # status 401
sed -n '/" 403 /p' access.log           # status 403
sed -n '\|/admin/login|p' access.log    # admin login page
sed -n '/POST/p' access.log             # POST requests
sed -n '/\.\.\//p' access.log           # path traversal ../
sed -n '/<script>/p' access.log         # XSS attempt
sed -n '/OR/p' access.log               # SQL injection
sed -n '/" 200 /!p' access.log          # everything except 200
sed -n '/^10\.0\.0\.25/p' access.log    # from one IP
```

Observation / Explanation: For the status codes I searched for the quote and spaces around the number, like " 200 ", instead of just 200. This matters because the byte count at the end of a line could also be 200 and would give a false match. For /admin/login I used \| to change the delimiter to a pipe, otherwise every forward slash in the path would need escaping. The ^ in the last command anchors the match to the start of the line so it only catches the source IP, not an IP appearing anywhere else.

8. Exercise 7 — Labelling Suspicious Web Requests

```
sed 's/ 200 / [ALLOWED] /' access.log
sed 's/ 401 / [UNAUTHORIZED] /' access.log
sed 's/ 403 / [BLOCKED] /' access.log
sed 's|/admin/login|/RESTRICTED_RESOURCE|' access.log
sed 's/GET/READ_REQUEST/' access.log
sed 's/POST/SUBMIT_REQUEST/' access.log
```

Observation / Explanation: I kept the spaces around the status numbers in both the search and the replacement so the line spacing doesn't get mangled. For the URL substitution I switched the separator from / to | sed lets you use almost any character there, and doing that is much cleaner than escaping the slashes in the path.

9. Exercise 8 — Grouping Security Operations with { }

```
# 401 entries: substitute then print
sed -n '/ 401 /{s/ 401 / [UNAUTHORIZED] /; p}' access.log

# 403 entries: substitute, redact the source IP, then print
sed -n '/ 403 /{s/ 403 / [BLOCKED] /; s/^[0-9.]\+/[REDACTED_IP]/; p}' access.log

# /admin/login: rename the URL, redact the IP, then print
sed -n '\|/admin/login|{s|/admin/login|/PROTECTED_PAGE|; s/^[0-9.]\+/[REDACTED_IP]/; p}' access.log
```

Observation / Explanation: The curly braces let you run several commands on the same matched line instead of making a separate pass for each one. The commands inside are separated by semicolons and they run in order, so the substitution happens first and then p prints the already-changed line. ^[0-9.]\+ means one or more digits or dots at the very start of the line, which is exactly the source IP field.

10. Exercise 9 — Adding Incident Labels

```
sed '1i SECURITY AUTHENTICATION LOG' auth.log        # insert before line 1
sed '$a END OF SECURITY LOG' auth.log                # append after last line
sed '/Failed password/i [WARNING]' auth.log          # insert before each match
sed '/Invalid user/a Review this event' auth.log     # append after each match
sed '/ 403 /i [SUSPICIOUS WEB REQUEST]' access.log
sed '/ 401 /a [REQUIRES INVESTIGATION]' access.log
sed '/Connection closed/c [CONNECTION TERMINATED]' auth.log
sed '/" 200 /c [NORMAL WEB REQUEST]' access.log
```

Observation / Explanation: There are three different commands here and it's easy to mix them up. i puts a new line before the match, a puts one after it, and c throws away the matched line completely and replaces it with the new text. So the last two commands don't just label those entries, they actually wipe out the original content of the line, which is why the question worded them as "replace all entries containing" rather than "label".

11. Exercise 10 — Separating Entries with G

```
sed 'G' auth.log            # blank line after every entry
sed '/Failed password/G' auth.log     # blank line after failed logins
sed '/ 403 /G' access.log             # blank line after blocked requests
sed '1~2G' auth.log                   # blank line after every 2nd entry
```

Observation / Explanation: G copies the hold space onto the end of the current line. The hold space is a second buffer that sed keeps aside, and since nothing was ever put into it here, it is empty so appending it just adds a newline and you get a blank line. It's a slightly odd way to double-space a file but it works, and it only affects the lines matching the address you give it.

12. Exercise 11 — Writing Events to Separate Files

```
sed -n '/Failed password/w failed_logins.txt' auth.log
sed -n '/Accepted password/w successful_logins.txt' auth.log
sed -n '/Invalid user/w invalid_users.txt' auth.log
sed -n '/" 401 /w unauthorized_requests.txt' access.log
sed -n '/" 403 /w blocked_requests.txt' access.log
sed -n '/172\.16\.0\.8/w suspicious_ip.txt' auth.log access.log

head -v failed_logins.txt successful_logins.txt invalid_users.txt
head -v unauthorized_requests.txt blocked_requests.txt suspicious_ip.txt
```

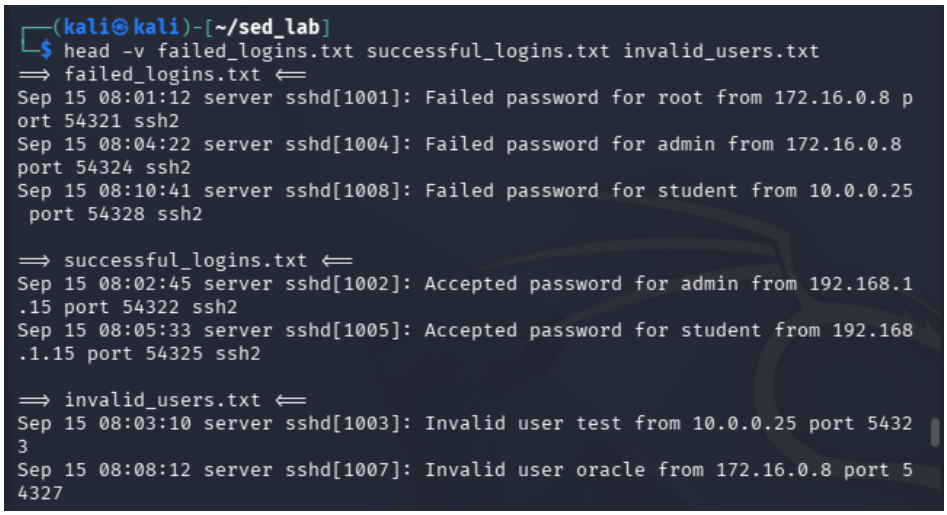


Figure 3: Contents of failed_logins.txt, successful_logins.txt and invalid_users.txt.

Observation / Explanation: The w command writes matching lines straight into a file instead of printing them. The split came out correctly — 3 failed passwords, 2 accepted passwords, and 2 invalid user attempts, which matches what is in the original auth.log. I used head -v so the filename gets shown above each block, otherwise you can't tell where one file ends and the next begins.

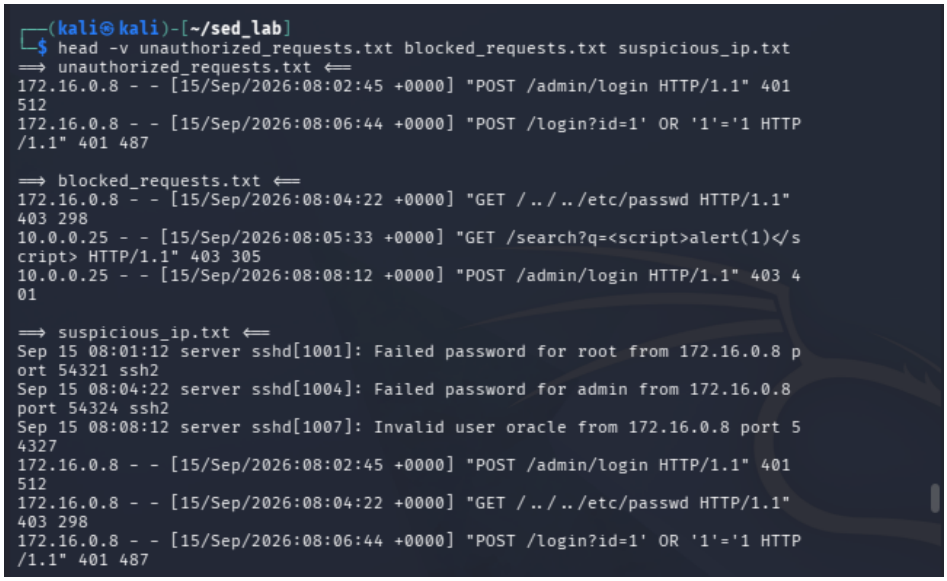


Figure 4: Contents of unauthorized_requests.txt, blocked_requests.txt and suspicious_ip.txt.

Observation / Explanation: The 401 file has 2 entries and the 403 file has 3, matching the sample access.log. The interesting one is suspicious_ip.txt — I passed both auth.log and access.log to that single command, so sed read them one after the other as one long stream and collected everything involving 172.16.0.8 from both. That's why it contains a mix of SSH login lines and web request lines, which is genuinely useful if you're tracking one attacker across different services.

13. Exercise 12 — Combining Operations

```
# label both login outcomes
sed -e 's/Accepted password/[SUCCESSFUL LOGIN]/' \
    -e 's/Failed password/[FAILED LOGIN]/' auth.log

# label both web statuses
sed -e 's/ 401 / [UNAUTHORIZED] /' -e 's/ 403 / [BLOCKED] /' access.log

# delete successful logins, keep only failed ones
sed -e '/Accepted password/d' -e '/Failed password/!d' auth.log

# label root and mask every IP
sed -E -e 's/root/[PRIVILEGED_USER]/g' \
       -e 's/([0-9]{1,3}\.){3}[0-9]{1,3}/[REDACTED_IP]/g' auth.log

# both HTTP methods
sed -e 's/GET/READ_REQUEST/' -e 's/POST/SUBMIT_REQUEST/' access.log
```

Observation / Explanation: Using multiple -e options lets sed apply all the commands in a single pass over the file, which is faster than piping one sed into another. The order does matter though the commands run top to bottom on each line, so if an earlier substitution changes text that a later one was looking for, the later one won't find it.

14. Exercise 13 — Using a sed Script File

Instead of typing a long command, all the commands go into a .sed file and sed runs them with the -f option.

```
cat > security_report.sed << 'EOF'
1i SECURITY INCIDENT REPORT
s/Failed password/[FAILED LOGIN]/
s/Invalid user/[INVALID USER ATTEMPT]/
s/Accepted password/[SUCCESSFUL LOGIN]/
s/[0-9]\{1,3\}\.[0-9]\{1,3\}\.[0-9]\{1,3\}\.[0-9]\{1,3\}/[REDACTED_IP]/g
$a END OF INCIDENT REPORT
EOF

sed -f security_report.sed auth.log
sed -f security_report.sed auth.log > incident_report.txt
```

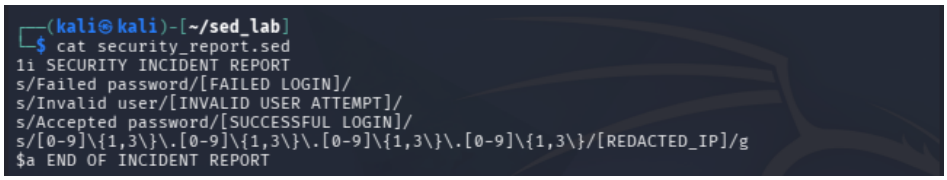


Figure 5: The contents of security_report.sed.

Observation / Explanation: One command per line, no quotes needed around anything because the shell never sees these — sed reads them straight from the file. The IP pattern looks messy because inside a script file you can't use the -E option, so it has to be written in basic regex with the braces escaped as \{1,3\}.

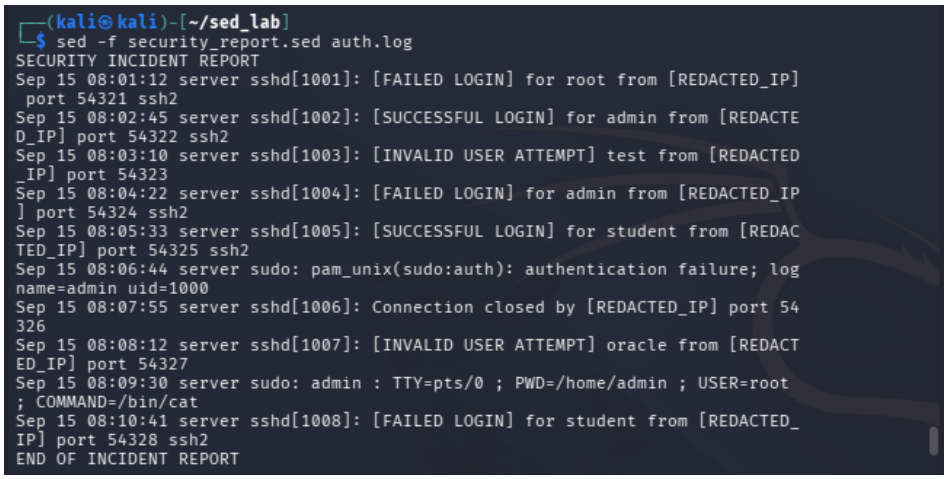


Figure 6: Running the script with sed -f security_report.sed auth.log.

Observation / Explanation: All six commands applied in one go. The heading got inserted at the top, every IP became [REDACTED_IP], the three login types got their labels, and the footer was appended at the bottom. You can see the substitutions worked on the right lines — line 6's authentication failure and line 9's sudo command were left alone because none of the patterns matched them.

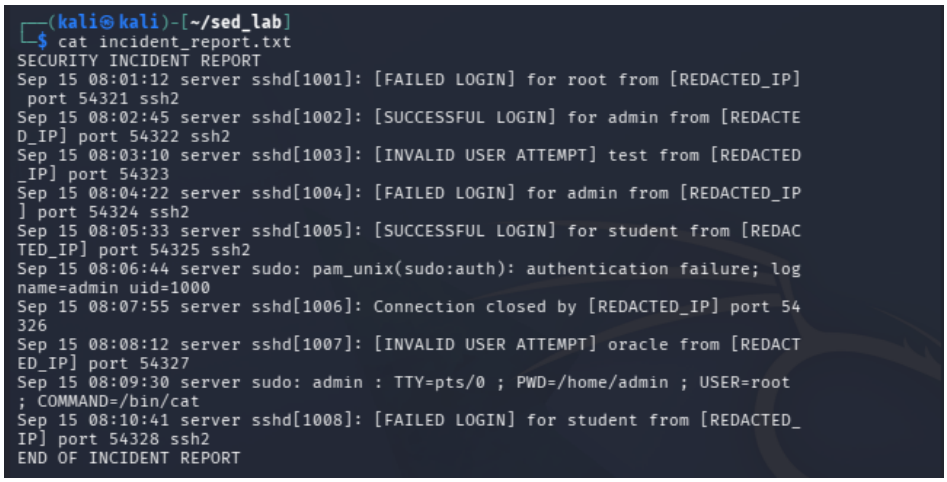


Figure 7: The saved output in incident_report.txt.

Observation / Explanation: Redirecting with > saved the same output into a file. The contents are identical to what was printed on screen, confirming the redirect captured everything including the inserted heading and appended footer.

15. Exercise 14 — Editing a Configuration File In Place

This is the only exercise where sed actually modifies the file, using the -i option. A backup was taken first.

```
cp security.conf security.conf.backup

sed -i 's/PermitRootLogin yes/PermitRootLogin no/' security.conf
sed -i 's/MaxAuthTries 6/MaxAuthTries 3/' security.conf
```

```
sed -i 's/FirewallStatus disabled/FirewallStatus enabled/' security.conf
sed -i 's/LogLevel INFO/LogLevel VERBOSE/' security.conf
sed -i 's/GuestLogin enabled/GuestLogin disabled/' security.conf

diff security.conf.backup security.conf

# safer form which makes its own backup
sed -i.bak 's/PasswordAuthentication yes/PasswordAuthentication no/' security.conf
```



Figure 8: security.conf before any changes.

Observation / Explanation: The starting file, with all six settings in their insecure state — root login allowed, 6 auth tries, firewall off, basic logging and guest login on.



Figure 9: security.conf after the five sed -i commands.

Observation / Explanation: All five settings changed as intended. Unlike every earlier exercise, the file on disk is genuinely different now -i writes the changes back instead of just printing them. PasswordAuthentication is still yes at this point because that one gets handled separately in the next step.



Figure 10: diff comparing the backup against the edited file.

Observation / Explanation: diff confirms exactly five lines changed and nothing else got touched. The < lines are from the backup and the > lines are the current file. This is why taking the backup first mattered — without it there'd be no way to verify the edits or undo them if a pattern had been wrong.

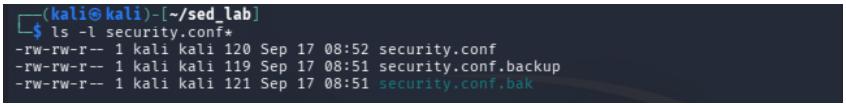


Figure 11: Listing the files after using sed -i.bak.

Observation / Explanation: There are now three files. security.conf.backup is the manual copy I made at the start, and security.conf.bak was created automatically by the -i.bak form. The sizes differ slightly because each holds a different stage of the edits.

```
┌──(kali㉿kali)-[~/sed_lab]
└─$ cat security.conf
PermitRootLogin no
PasswordAuthentication no
MaxAuthTries 3
FirewallStatus enabled
LogLevel VERBOSE
GuestLogin disabled
```

Figure 12: security.conf after the -i.bak edit.

Observation / Explanation: PasswordAuthentication is now no as well, so all six settings have been hardened. The -i.bak form is the safer habit — it does the same in-place edit but keeps a copy of the previous version without you having to remember to run cp first.

16. Exercise 15 — security_log_analyser.sh

The final exercise pulls everything together into one script that takes a log file, checks it, filters out the suspicious entries, masks the IPs, and writes a labelled report.

```
#!/bin/bash

if [ $# -eq 0 ]; then
    echo "Error: No log file provided."
    echo "Usage: $0 <logfile>"
    exit 1
fi

logfile="$1"

if [ ! -e "$logfile" ]; then
    echo "Error: File '$logfile' does not exist."
    exit 1
fi

if [ ! -r "$logfile" ]; then
    echo "Error: File '$logfile' is not readable."
    exit 1
fi

output="security_alert_report.txt"

echo "SECURITY ALERT REPORT" > "$output"

sed -n -e '/Failed password/p' -e '/Invalid user/p' "$logfile" \
  | sed -E -e 's/([0-9]{1,3}\.){3}[0-9]{1,3}/[REDACTED_IP]/g' \
           -e 's/^/[ALERT] /' >> "$output"

echo "END OF REPORT" >> "$output"

echo "Report saved to $output"
echo
cat "$output"
```

```
┌──(kali㉿kali)-[~/sed_lab]
└─$ ./security_log_analyser.sh
Error: No log file provided.
Usage: ./security_log_analyser.sh <logfile>
```

Figure 13: Running the script with no argument.

Observation / Explanation: $# is 0 so it prints the usage message and exits straight away, before trying to use an argument that isn't there.



Figure 14: Running the script against a file that does not exist.

Observation / Explanation: nofile.log fails the -e check, so the script stops there and never gets as far as running any sed commands on a file that isn't present.

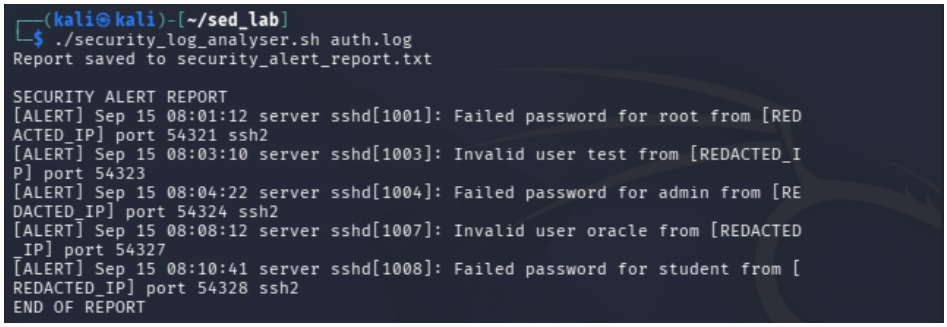


Figure 15: The full generated report for auth.log.

Observation / Explanation: This is the whole thing working. The first sed pulled out just the 3 failed passwords and 2 invalid user attempts, ignoring the successful logins and sudo lines. That output was piped into a second sed which masked every IP as [REDACTED_IP] and stuck [ALERT] on the front of each line using s/^/[ALERT] /, since ^ matches the start of a line and substituting there just prefixes text. The heading and footer came from plain echo commands, with > creating the file and >> adding to it. Five alert lines in total, which is correct.

17. Conclusion

The main thing I took away from this lab is that sed is safe by default. Every exercise except the config file one left the original log completely untouched, which matters a lot when you're working with evidence during an investigation. Escaping the dots in IP addresses turned out to be the easiest thing to get wrong, and searching for status codes needed the quotes and spaces included so the byte count at the end of a line wouldn't cause a false match. The most useful parts were the w command for splitting one log into separate files by event type, and the script file in Exercise 13, which meant I could run a whole set of substitutions repeatedly without retyping them. Exercise 15 tied it together by combining the argument and file checks from the previous assignment with two piped sed commands to produce a finished report.